

UNIVERSITÉ ABDELMALEK ESSAADI

Faculté des Sciences et Techniques – Tanger

Département d'Informatique

MASTER SCIENCES ET TECHNIQUES INTELLIGENCE ARTIFICIELLE ET SCIENCES DES DONNÉES

Module : Big Data

Rapport – Mini-Projet Big Data

Analyse des Avis Clients en Temps Réel
sur des Produits Amazon

Encadré par :

Pr. El Yusufi Yasyn

Réalisé par :

El Attari Taki Eddine

Chichaoui Oussama

Bouazza Mohamed

El Kajdouhi Mohamed Ayman

Année Universitaire 2025–2026

Table des matières

Introduction Générale	5
1 Données et Problématique	6
1.1 Source des données	6
1.2 Définition de la cible	6
1.3 Partitionnement des données	7
2 Architecture du Système	8
2.1 Vue d'ensemble de l'architecture	8
2.2 Technologies utilisées	8
2.3 Rôle de Kafka dans l'architecture	9
2.4 Services Docker	9
3 Préparation des Données et Entraînement du Modèle	10
3.1 Exploration des données (EDA)	10
3.2 Prétraitement du texte	10
3.3 Pipeline ML avec Spark MLlib	10
3.4 Évaluation du modèle	11
4 Pipeline de Streaming Temps Réel	12
4.1 Kafka Producer	12
4.2 Spark Consumer – Lecture et Prédiction	12
4.2.1 Chargement du modèle	12
4.2.2 Lecture du topic Kafka	13
4.3 Chemin 1 – Visualisation en Continu (Live Feed)	13
4.3.1 Écriture vers Kafka <code>predictions-live</code>	13
4.3.2 Consommation par Django via WebSocket	13
4.4 Chemin 2 – Analyses Statistiques (Dashboard)	14
4.4.1 Écriture vers MongoDB	14
4.4.2 Aggregation MongoDB	14
5 Interface Web et Résultats	15
5.1 Page de Connexion	15
5.2 Pipeline Control	16
5.3 Live Feed – Visualisation en Continu	16
5.4 MongoDB Compass – Base de Données	17
5.5 Analytics Dashboard	17
5.6 Product Detail – Analyse par Produit	18

6	APIs REST et WebSocket	20
6.1	Endpoints REST	20
6.2	Format des messages WebSocket	20
7	Supervision avec Apache Airflow	21
7.1	Rôle et positionnement d’Airflow	21
7.2	Architecture du DAG <code>pipeline_watchdog</code>	21
7.3	Description des 7 tâches de vérification	22
7.4	Gestion de la phase de démarrage — <code>AirflowSkipException</code>	22
7.5	DAG Airflow – Capture d’écran	23
7.6	Installation et lancement	24
	Conclusion	25

Table des figures

5.1	Page de connexion – Interface <i>Pulpe Sentiment LIVE</i>	15
5.2	Page Pipeline Control – Kafka Producer et Spark Consumer actifs	16
5.3	Page Live Feed – Flux temps réel des prédictions via WebSocket	16
5.4	MongoDB Compass – Collection <code>reviews_db.predictions</code>	17
5.5	Analytics Dashboard – KPI et tendance annuelle des sentiments	17
5.6	Product Detail – Distribution des sentiments pour un produit	18
5.7	Product Detail – Confidence Distribution, comparaison et liste des avis . .	18
7.1	DAG <code>pipeline_watchdog</code> dans l'interface Airflow – Vue Graph	23

Liste des tableaux

1.1	Description du dataset Amazon Fine Food Reviews	6
1.2	Partitionnement du dataset	7
2.1	Stack technique du projet	8
2.2	Services Docker Compose	9
3.1	Métriques de performance du modèle (jeu de test)	11
3.2	Performances par classe	11
6.1	APIs REST Django	20
7.1	Tâches du DAG <code>pipeline_watchdog</code>	22
7.2	Phases de comportement d’Airflow au démarrage	23

Introduction Générale

Dans un contexte où les plateformes de commerce électronique génèrent des millions d'avis clients chaque jour, l'analyse automatique du sentiment de ces avis constitue un enjeu stratégique majeur. Amazon, l'une des plus grandes plateformes mondiales, met à disposition un jeu de données riche de plus de 568 000 avis alimentaires, offrant une opportunité unique pour mettre en œuvre un pipeline de traitement de données massives en temps réel.

Ce mini-projet, réalisé dans le cadre du module **Big Data** de notre Master IASD, s'inscrit dans cette problématique. L'objectif est de concevoir une architecture de bout en bout capable de :

- **Ingérer** un flux continu d'avis Amazon via **Apache Kafka**, simulant un véritable flux de données en temps réel.
- **Traiter et classifier** chaque avis grâce à **Apache Spark Structured Streaming** et un modèle de *machine learning* entraîné avec **Spark MLlib**.
- **Stocker** les résultats de prédiction dans **MongoDB**, une base de données NoSQL adaptée aux flux massifs.
- **Visualiser** les résultats en deux modes : un flux temps réel via **WebSocket** et un tableau de bord analytique via **REST API**.
- **Superviser** l'ensemble du pipeline grâce à **Apache Airflow**, jouant le rôle d'observateur externe indépendant (*watchdog*) qui vérifie en continu la santé de chaque composant.

Le pipeline produit ainsi deux sorties complémentaires : un flux *online* présentant chaque prédiction au fil de l'eau avec une latence inférieure à 2 secondes, et un tableau de bord *offline* proposant des analyses statistiques enrichies (tendances annuelles, distribution des sentiments, analyse par produit). Un DAG Airflow surveille l'ensemble toutes les 5 minutes, garantissant la détection précoce de toute panne silencieuse.

Ce rapport documente l'ensemble des étapes de réalisation : de l'exploration des données et l'entraînement du modèle, en passant par la mise en place du pipeline de streaming et du système de supervision Airflow, jusqu'au déploiement de l'interface web complète.

1. Données et Problématique

1.1 Source des données

Le dataset utilisé provient de **Kaggle – Amazon Fine Food Reviews**¹. Il regroupe les avis sur des produits alimentaires disponibles sur Amazon sur une période de plus de 10 ans (2002–2012).

Table 1.1: Description du dataset Amazon Fine Food Reviews

Champ	Type	Description
Id	Entier	Identifiant unique de l’avis
ProductId	Chaîne	Identifiant unique du produit
UserId	Chaîne	Identifiant unique de l’utilisateur
ProfileName	Chaîne	Pseudo de l’utilisateur
Score	Entier (1–5)	Note donnée au produit
Time	Timestamp Unix	Date de l’avis
Summary	Texte	Résumé court de l’avis
Text	Texte	Contenu complet de l’avis

1.2 Définition de la cible

La variable cible (sentiment) est construite à partir du score numérique selon la règle suivante :

Règle de labellisation

- **Score** < **3** → Sentiment **Négatif** (classe 0)
- **Score** = **3** → Sentiment **Neutre** (classe 1)
- **Score** > **3** → Sentiment **Positif** (classe 2)

¹<https://www.kaggle.com/datasets/snap/amazon-fine-food-reviews>

1.3 Partitionnement des données

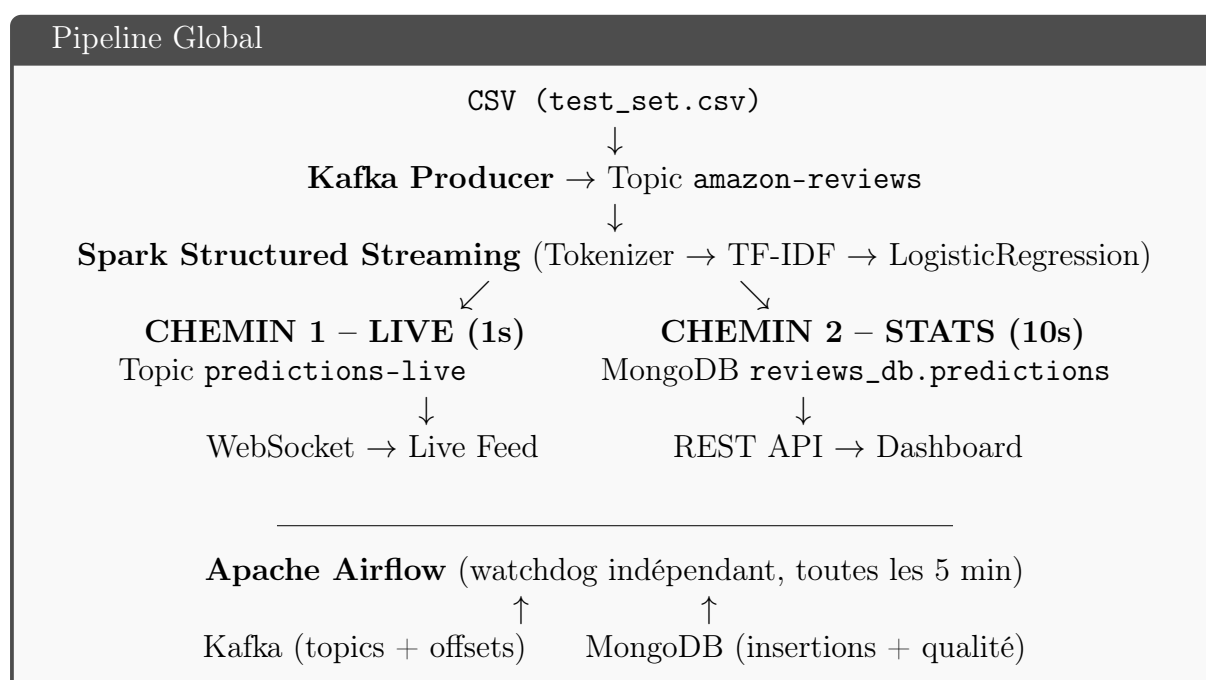
Table 1.2: Partitionnement du dataset

Ensemble	Proportion	Usage
Entraînement (<code>train.csv</code>)	80%	Entraînement du modèle
Validation (<code>val.csv</code>)	10%	Ajustement des hyperparamètres
Test (<code>test_set.csv</code>)	10%	Évaluation finale + streaming
Total	100%	568 454 avis

2. Architecture du Système

2.1 Vue d'ensemble de l'architecture

L'architecture globale du projet s'articule autour d'un pipeline de streaming en deux chemins parallèles, comme illustré dans le schéma suivant :



2.2 Technologies utilisées

Table 2.1: Stack technique du projet

Composant	Technologie
Message Broker	Apache Kafka 7.5 (Confluent) + Zookeeper
Stream Processing	Apache Spark 3.5 (Structured Streaming)
Machine Learning	Spark MLlib – Pipeline (Tokenizer → TF-IDF → LR)
Base de données (prédictions)	MongoDB 7
Base de données (auth/méta)	PostgreSQL 16
Backend	Django 4.2 + Django Channels 4 + Daphne ASGI
Frontend	React 18.3 (SPA) + Chart.js 4.4
Supervision / Monitoring	Apache Airflow 2.9 (DAG <i>pipeline_watchdog</i>)
Containerisation	Docker Compose (6 services)

2.3 Rôle de Kafka dans l'architecture

Apache Kafka joue un double rôle dans notre architecture :

1. **Entrée** : Le producer injecte les avis du CSV de test dans le topic `amazon-reviews` au rythme de 2 reviews par seconde (délai 0.5s), simulant un flux de données Amazon en temps réel.
2. **Sortie** : Spark publie les prédictions dans le topic `predictions-live` toutes les secondes. Ce choix de Kafka comme bus de sortie découple complètement Spark et Django : chaque composant fonctionne indépendamment, ce qui est l'architecture standard des systèmes de streaming en production.

Chaque broker Kafka gère une partition d'un topic donné. La publication et la souscription du flux de commentaires s'effectuent à travers ces topics, garantissant une haute disponibilité et une tolérance aux pannes.

2.4 Services Docker

L'ensemble du projet est containerisé via Docker Compose en 6 services :

Table 2.2: Services Docker Compose

Service	Image	Port exposé
Zookeeper	Confluent Zookeeper	2181
Kafka Broker	Confluent Kafka	29092
MongoDB	MongoDB 7	27018
PostgreSQL	PostgreSQL 16	5433
Kafka UI	Provectus Kafka-UI	8081
Mongo Express	Mongo-Express	8082

3. Préparation des Données et Entraînement du Modèle

3.1 Exploration des données (EDA)

Le notebook 01_eda.ipynb permet d'analyser la distribution initiale des sentiments, les longueurs de texte, et les fréquences de mots. On observe que les avis positifs (score 4–5) sont très largement majoritaires dans le dataset, ce qui crée un déséquilibre de classes à prendre en compte lors de l'entraînement.

3.2 Prétraitement du texte

Le notebook 02_preprocessing.ipynb implémente une chaîne de nettoyage complète :

```
def clean_text(text):
    text = str(text).lower()
    text = re.sub(r'<.*?>', ' ', text)          # retire les balises
    HTML
    text = re.sub(r'http\S+', ' ', text)        # retire les URLs
    text = re.sub(r'[^a-z\s]', ' ', text)        # garde seulement les
    lettres
    tokens = [lemmatizer.lemmatize(t)
               for t in text.split()
               if t not in stop_words and len(t) > 2]
    return ' '.join(tokens)
```

Listing 3.1: Fonction de nettoyage du texte

Exemple concret : "This product is NOT good!!! I hate it :(")

→ après nettoyage : "product good hate"

Le champ d'entrée du modèle est la concaténation de **Summary** et **Text**, ce qui enrichit le contexte disponible pour la classification.

3.3 Pipeline ML avec Spark MLlib

Le notebook 03_training.ipynb construit un pipeline Spark ML complet :

```
pipeline = Pipeline(stages=[
    Tokenizer(inputCol="cleaned", outputCol="tokens"),
    StopWordsRemover(inputCol="tokens", outputCol="filtered"),
    HashingTF(inputCol="filtered", outputCol="rawFeatures",
               numFeatures=20000),
    IDF(inputCol="rawFeatures", outputCol="features", minDocFreq=5)
])
```

```
LogisticRegression(featuresCol="features", labelCol="sentiment"
,
maxIter=100, regParam=0.01),
])
```

Listing 3.2: Pipeline Spark MLlib

Optimisation des hyperparamètres : une *CrossValidation* à 3 folds est utilisée pour sélectionner la meilleure combinaison de paramètres. Le meilleur modèle est sauvegardé via `cv_model.bestModel.save("models/best_model")` – il est important de sauvegarder `bestModel` (un `PipelineModel`) et non `cv_model` (un `CrossValidatorModel`) pour pouvoir le recharger avec `PipelineModel.load()`.

3.4 Évaluation du modèle

Le notebook `04_evaluation.ipynb` évalue les performances sur le jeu de test (10%) :

Table 3.1: Métriques de performance du modèle (jeu de test)

Métrique	Valeur
F1-score macro (test)	0.8320
Accuracy (test)	80.77%
Precision	87.50%
Recall	80.77%

Table 3.2: Performances par classe

Classe	Précision	Recall	F1
Négatif	0.85	0.83	0.84
Neutre	0.43	0.37	0.40
Positif	0.92	0.93	0.93

Analyse des résultats

La classe **Neutre** présente des performances plus faibles ($F1 = 0.40$). Cela s'explique par le fait que les avis à 3 étoiles sont intrinsèquement ambigus et sous-représentés dans le dataset. Ce phénomène est courant dans les problèmes NLP de sentiment à 3 classes sur des données déséquilibrées.

4. Pipeline de Streaming Temps Réel

4.1 Kafka Producer

Le fichier `kafka_producer/producer.py` lit le CSV de test et envoie chaque avis dans le topic Kafka `amazon-reviews` au format JSON avec un délai de 0.5 seconde entre chaque message (2 reviews/seconde).

```
producer = KafkaProducer(
    bootstrap_servers="localhost:29092",
    value_serializer=lambda v: json.dumps(v).encode('utf-8'),
)
for _, row in df.iterrows():
    msg = {
        "Id": int(row["Id"]),
        "ProductId": str(row["ProductId"]),
        "Summary": str(row.get("Summary", "")),
        "Text": str(row.get("Text", "")),
        "cleaned": str(row.get("cleaned", "")),
        "true_sentiment": int(row["sentiment"]),
    }
    producer.send("amazon-reviews", key=str(row["ProductId"]),
                  value=msg)
    time.sleep(0.5)
```

Listing 4.1: Kafka Producer – envoi des reviews

4.2 Spark Consumer – Lecture et Prédiction

Le fichier `spark_streaming/consumer.py` constitue le cœur du pipeline. Il lit le topic Kafka en Structured Streaming et applique le modèle ML.

4.2.1 Chargement du modèle

```
from pyspark.ml import PipelineModel
model = PipelineModel.load("./models/best_model")
# Contient : Tokenizer -> StopWordsRemover -> HashingTF -> IDF ->
LR
```

Listing 4.2: Chargement du modèle PipelineModel

4.2.2 Lecture du topic Kafka

```
raw = spark.readStream \
    .format("kafka") \
    .option("kafka.bootstrap.servers", "localhost:29092") \
    .option("subscribe", "amazon-reviews") \
    .option("startingOffsets", "latest") \
    .load()

parsed = raw.select(
    from_json(col("value").cast("string"), schema).alias("d")
).select("d.*")
```

Listing 4.3: Lecture du stream Kafka

4.3 Chemin 1 – Visualisation en Continu (Live Feed)

4.3.1 Écriture vers Kafka predictions-live

Spark écrit les prédictions dans le topic predictions-live avec un trigger de 1 seconde :

```
predictions_df \
    .select(to_json(struct("id", "productId", "summary",
                          "sentiment", "confidence", "time")))
    .alias("value")) \
    .writeStream \
    .format("kafka") \
    .option("kafka.bootstrap.servers", KAFKA_BROKER) \
    .option("topic", "predictions-live") \
    .trigger(processingTime="1 second") \
    .start()
```

Listing 4.4: Stream vers Kafka predictions-live

4.3.2 Consommation par Django via WebSocket

Un thread Django (kafka_live_consumer.py) consomme le topic en permanence et diffuse chaque prédiction via Django Channels :

```
async_to_sync(channel_layer.group_send)(
    "live_feed",
    {"type": "live.batch", "predictions": [prediction_data]}
)
```

Listing 4.5: Thread Django KafkaConsumer

La latence bout-en-bout est de **1 à 2 secondes**, depuis la publication par Spark jusqu'à l'affichage dans le navigateur via WebSocket.

4.4 Chemin 2 – Analyses Statistiques (Dashboard)

4.4.1 Écriture vers MongoDB

Spark insère les prédictions dans MongoDB via `foreachBatch` toutes les 10 secondes :

```
def write_to_mongo(batch_df, batch_id):
    docs = batch_df.toPandas().to_dict("records")
    if docs:
        get_collection().insert_many(docs)

predictions_df.writeStream \
    .foreachBatch(write_to_mongo) \
    .trigger(processingTime="10 seconds") \
    .start()
```

Listing 4.6: Insertion dans MongoDB

4.4.2 Aggregation MongoDB

Django exécute des pipelines d'agrégation MongoDB pour produire les statistiques :

```
def get_yearly_trend():
    pipeline = [
        {"$addFields": {"year": {"$year": {"$toDate":
            {"$multiply": ["$Time", 1000]}}}}}},
        {"$group": {
            "_id": "$year",
            "positive": {"$sum": {"$cond":
                [{"$eq": ["$sentiment_label", "positive"]}, 1, 0]}},
            "neutral": {"$sum": {"$cond":
                [{"$eq": ["$sentiment_label", "neutral"]}, 1, 0]}},
            "negative": {"$sum": {"$cond":
                [{"$eq": ["$sentiment_label", "negative"]}, 1, 0]}},
        }},
        {"$sort": {"_id": 1}},
    ]
    return list(get_collection().aggregate(pipeline))
```

Listing 4.7: Aggregation MongoDB – tendance annuelle

5. Interface Web et Résultats

5.1 Page de Connexion

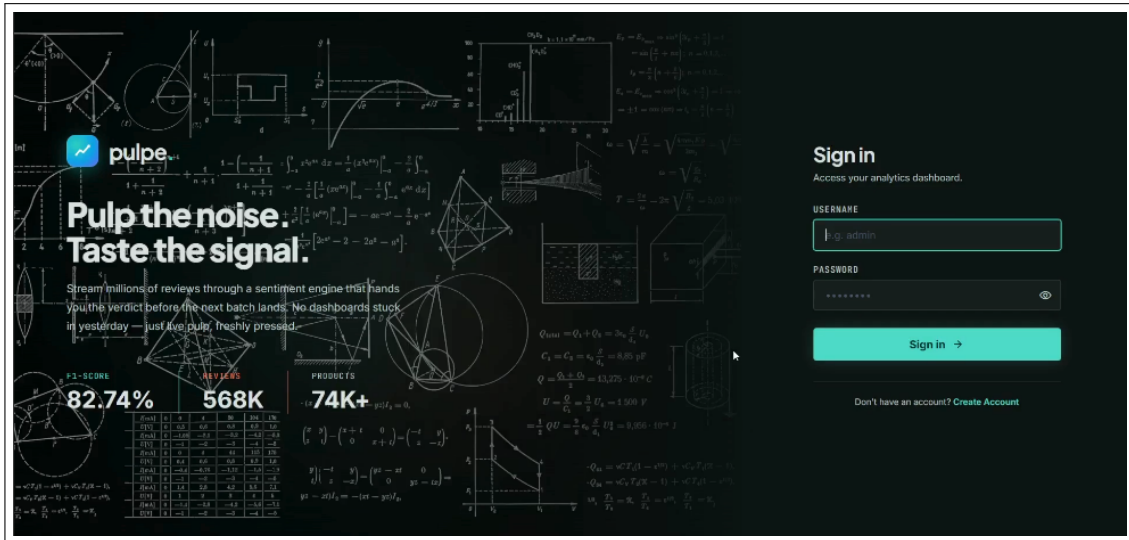


Figure 5.1: Page de connexion – Interface *Pulpe Sentiment LIVE*

Page d'accueil et d'authentification de l'application. L'interface est nommée *Pulpe* et affiche les métriques clés du modèle (F1-score : 82.74%, 568K reviews, 74K+ produits). La zone de droite présente le formulaire de connexion avec champs Username et Password.

5.2 Pipeline Control

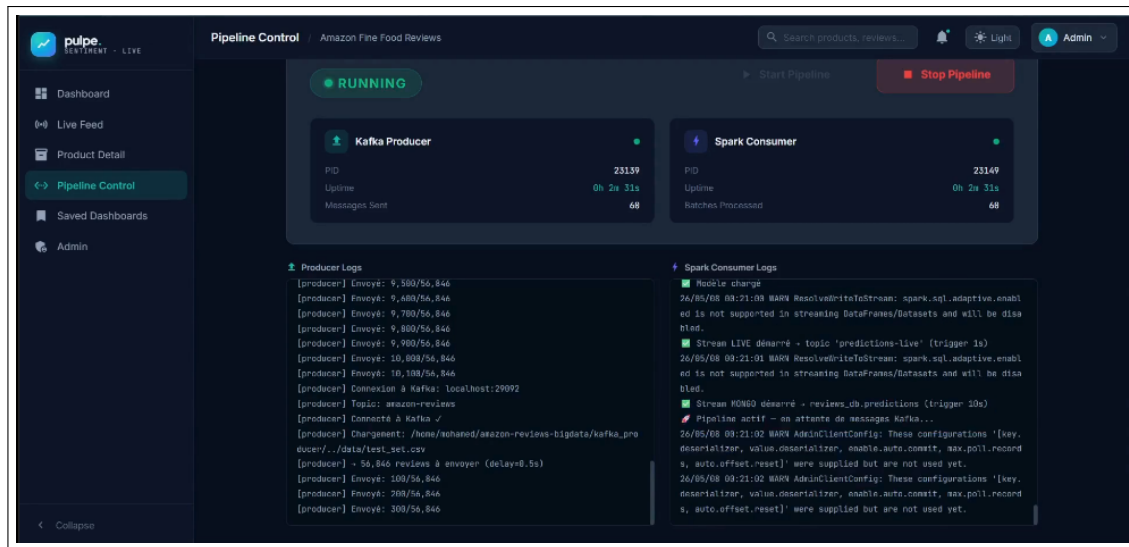


Figure 5.2: Page Pipeline Control – Kafka Producer et Spark Consumer actifs

Interface de contrôle du pipeline. Le statut indique **RUNNING**. On observe les PIDs du Kafka Producer (23139) et du Spark Consumer (23149), leur uptime et le nombre de messages envoyés (68 batches). Les logs de chaque composant sont affichés en temps réel.

5.3 Live Feed – Visualisation en Continu

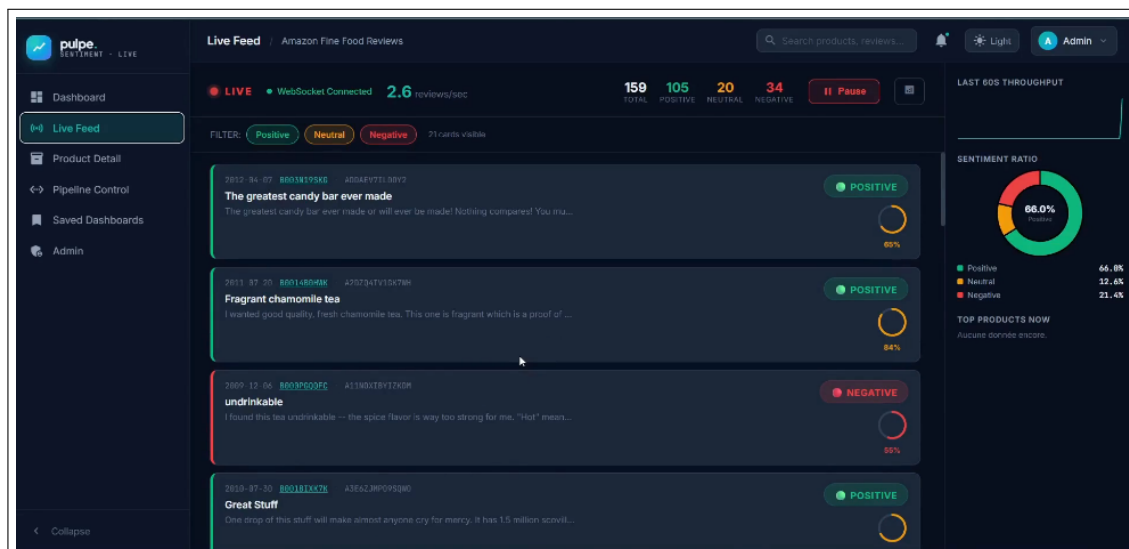


Figure 5.3: Page Live Feed – Flux temps réel des prédictions via WebSocket

Flux en temps réel des prédictions de sentiment. Chaque carte présente l'avis, son sentiment (POSITIVE / NEGATIVE) et le score de confiance. Le panneau latéral affiche le throughput (2.6 reviews/sec), les compteurs cumulés (159 total : 105 positifs, 20 neutres, 34 négatifs) et la répartition en camembert (66% positif, 12.6% neutre, 21.4% négatif). Les filtres permettent d'isoler une classe de sentiment.

5.4 MongoDB Compass – Base de Données

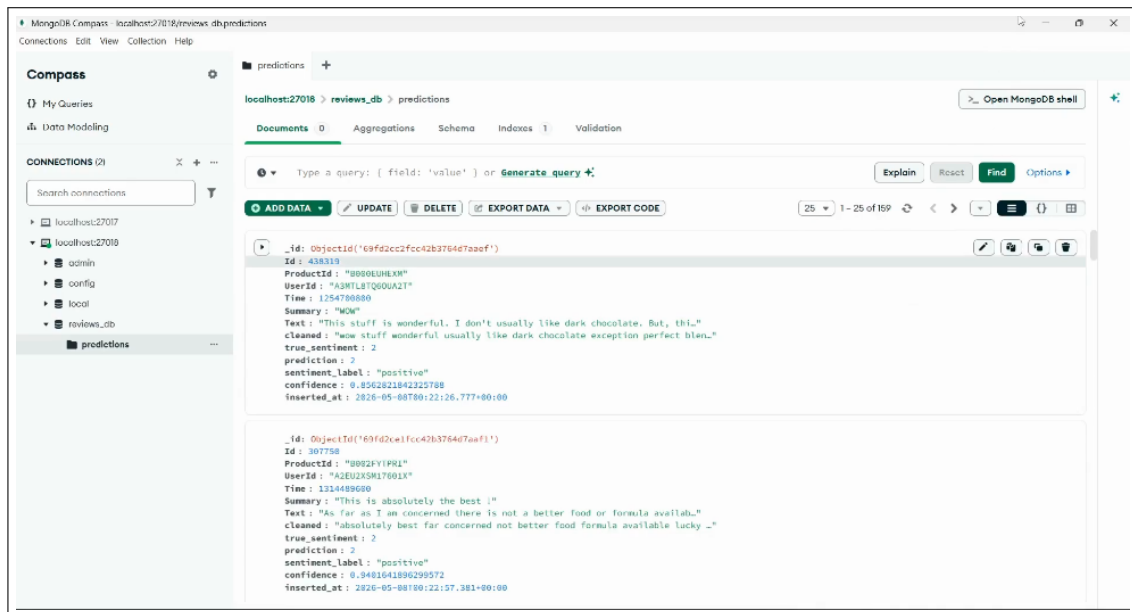


Figure 5.4: MongoDB Compass – Collection reviews_db.predictions

Vue MongoDB Compass sur la collection predictions de la base reviews_db. Chaque document stocke : l'Id de la review, le ProductId, l'UserId, le texte brut et nettoyé, la true_sentiment, la prediction (0/1/2), le sentiment_label ("positive"/"neutral"/"negative"), la confidence (ex. : 0.856) et le timestamp d'insertion (inserted_at).

5.5 Analytics Dashboard

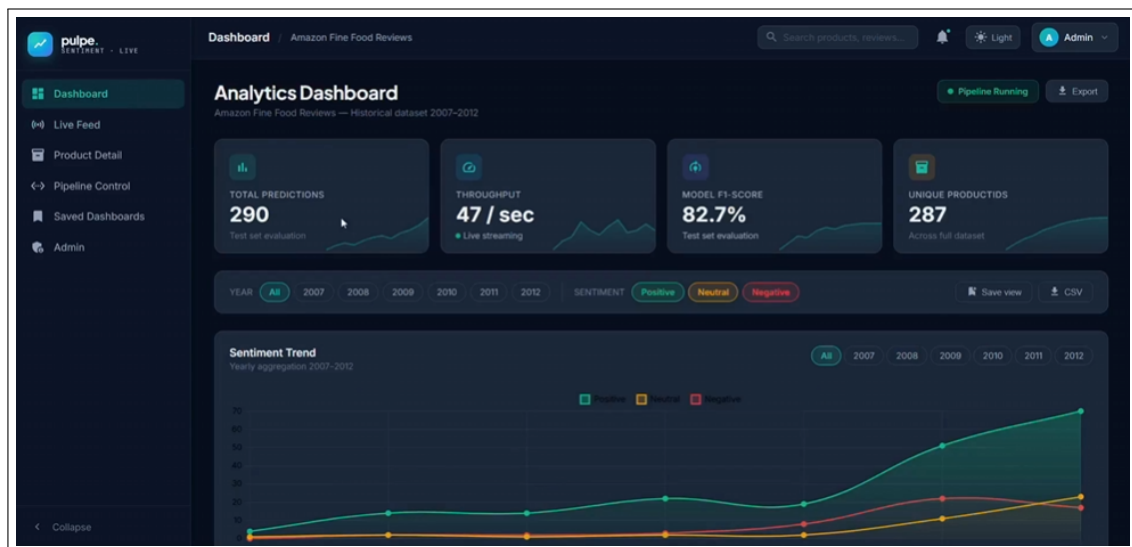


Figure 5.5: Analytics Dashboard – KPI et tendance annuelle des sentiments

Tableau de bord analytique avec 4 indicateurs clés (KPI) : Total Predictions (290), Throughput (47/sec), Model F1-Score (82.7%) et Unique Products (287). Le graphique

de tendance annuelle (Sentiment Trend 2007–2012) montre l'évolution des trois classes de sentiment au fil du temps avec Chart.js. Des filtres par année et par classe sont disponibles.

5.6 Product Detail – Analyse par Produit

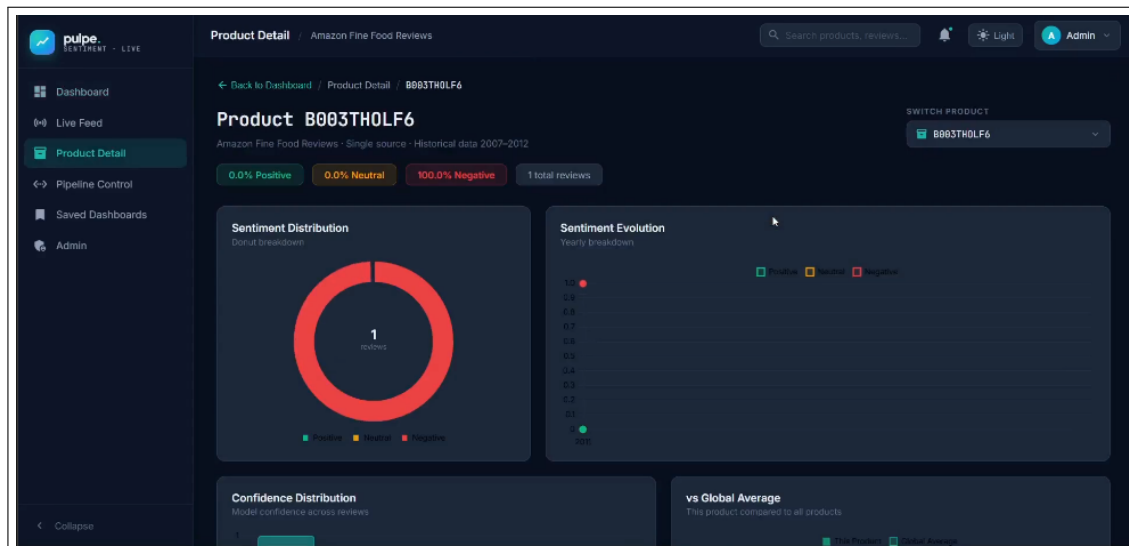


Figure 5.6: Product Detail – Distribution des sentiments pour un produit

Page de détail pour le produit B003TH0LF6. L'interface affiche la répartition des sentiments sous forme de donut chart (100% négatif pour ce produit avec 1 review) et l'évolution temporelle des sentiments (Sentiment Evolution). La navigation entre produits est possible via la liste déroulante *Switch Product*.

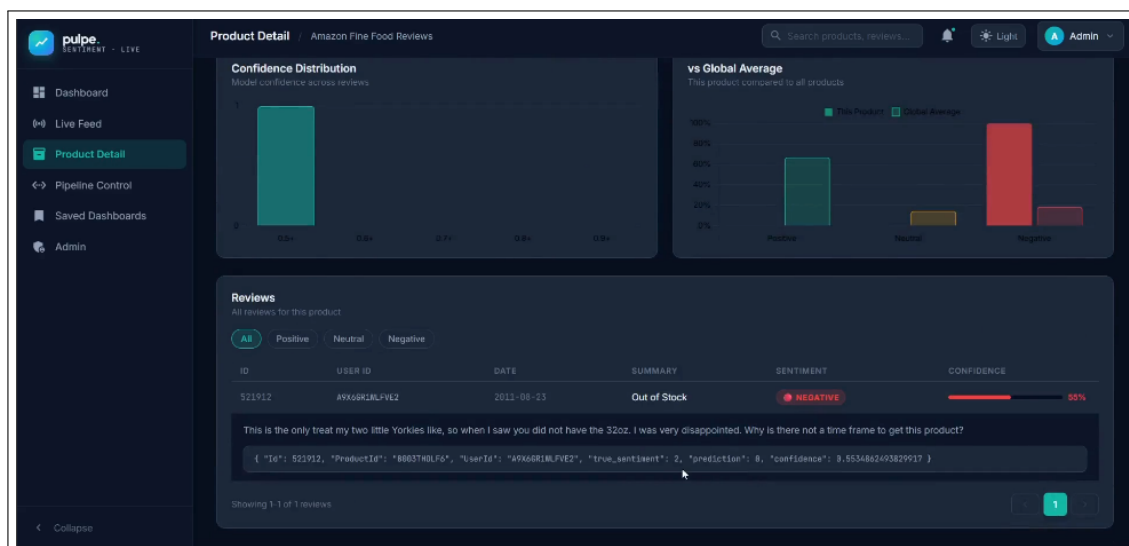


Figure 5.7: Product Detail – Confidence Distribution, comparaison et liste des avis

Suite de la page Product Detail. On y trouve : le graphique *Confidence Distribution* (histogramme des scores de confiance), la comparaison *vs Global Average* (barres comparant

ce produit aux moyennes globales), et la table des reviews filtrables (All/Positive/Neutral/Negative) avec les colonnes ID, User ID, Date, Summary, Sentiment et Confidence.

6. APIs REST et WebSocket

6.1 Endpoints REST

Table 6.1: APIs REST Django

Méthode	Endpoint	Description
GET	/api/kpi	Total prédictions, throughput, F1, produits
GET	/api/distribution	Répartition positive / neutre / négative
GET	/api/trend?year=All	Évolution annuelle des sentiments
GET	/api/recent?limit=50	Dernières prédictions
GET	/api/products/top	Top produits par sentiment
POST	/api/pipeline/start	Démarrer producer + spark-submit
POST	/api/pipeline/stop	Arrêter le pipeline
WS	/ws/live/	Flux temps réel des prédictions

6.2 Format des messages WebSocket

Le navigateur reçoit deux types de messages WebSocket :

- **init** : envoyé à la connexion, contient les 20 dernières prédictions depuis MongoDB.
- **batch** : envoyé à chaque nouvelle prédiction depuis Kafka, contient la review + les compteurs mis à jour.

```
{
  "type": "batch",
  "predictions": [{
    "id": "12345",
    "productId": "B001E4KFG0",
    "summary": "Good Quality Dog Food",
    "sentiment": "positive",
    "confidence": 0.94,
    "time": "1303862400"
  }],
  "counters": {"positive": 38415, "neutral": 9235, "negative": 8125},
  "source": "kafka_direct"
}
```

Listing 6.1: Format d'un message batch WebSocket

7. Supervision avec Apache Airflow

7.1 Rôle et positionnement d’Airflow

Apache Airflow a été intégré au projet en tant que **watchdog externe indépendant**. Son rôle n’est pas de contrôler ou de démarrer les composants du pipeline, mais de les *observer* passivement : il lit directement Kafka et MongoDB pour vérifier que tout fonctionne correctement, sans aucune interaction avec le backend Django.

Ce qu’Airflow fait et ne fait pas

Ce qu’Airflow NE fait PAS :

- × Ne démarre pas le producer
- × Ne démarre pas Spark
- × N’appelle aucune API Django
- × N’est pas importé dans le backend

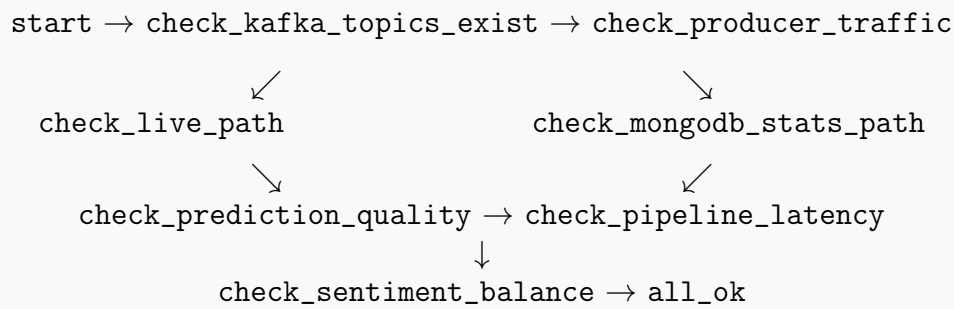
Ce qu’Airflow FAIT :

- ✓ Vérifie que les 2 topics Kafka existent
- ✓ Vérifie le trafic du producer
- ✓ Vérifie que Spark écrit dans `predictions-live`
- ✓ Vérifie les insertions MongoDB
- ✓ Vérifie la qualité des prédictions
- ✓ Vérifie la latence bout-en-bout (< 60s)
- ✓ Vérifie l’équilibre des sentiments

7.2 Architecture du DAG `pipeline_watchdog`

Le DAG s’exécute toutes les 5 minutes selon l’ordonnancement `* / 5 * * * *`. Il enchaîne 9 tâches selon le graphe de dépendances suivant :

Graphe des tâches du DAG



7.3 Description des 7 tâches de vérification

Table 7.1: Tâches du DAG pipeline_watchdog

Tâche	Ce qu'elle vérifie	Source
check_kafka_topics_exist	Les 2 topics <code>amazon-reviews</code> et <code>predictions-live</code> existent	Kafka
check_producer_traffic	L'offset de <code>amazon-reviews</code> croît (producer actif)	Kafka
check_live_path	L'offset de <code>predictions-live</code> croît (Spark actif)	Kafka
check_mongodb_stats_path	MongoDB reçoit de nouvelles insertions dans les 2 dernières min	MongoDB
check_prediction_quality	Les 50 derniers docs ont <code>sentiment_label</code> non null et confiance > 0.5	MongoDB
check_pipeline_latency	La dernière insertion a moins de 60s (<code>inserted_at</code>)	MongoDB
check_sentiment_balance	Pas plus de 95% positif ni 60% négatif (cohérence du modèle)	MongoDB

7.4 Gestion de la phase de démarrage — `AirflowSkipException`

Un point clé de l'implémentation est l'utilisation de `AirflowSkipException` pour gérer la période avant le lancement du pipeline. Plutôt que de marquer les tâches en FAIL rouge (ce qui serait un faux négatif), les tâches inapplicables passent en rose (*skipped*) :

```

def check_kafka_topics(**ctx):
    result = subprocess.run(
        ["docker", "exec", "kafka",
         "kafka-topics.sh", "--bootstrap-server",
         "localhost:9092", "--list"],
        capture_output=True, text=True, timeout=15
    )
    existing = set(result.stdout.strip().split('\n'))
    required = {"amazon-reviews", "predictions-live"}
  
```

```

missing = required - existing

if missing:
    raise AirflowSkipException(
        f"Pipeline pas encore lance -- topics manquants : {
            missing}"
    ) # Rose dans l'UI, pas d'echec

print(f"Topics Kafka OK : {required} presents")

```

Listing 7.1: Utilisation d'AirflowSkipException pour le démarrage

Table 7.2: Phases de comportement d'Airflow au démarrage

Run	Phase	UI Airflow
#1 et #2 (avant lancement)	Observation passive	Rose (Skipped) – zéro FAIL
#3 (après lancement)	Enregistrement des références	Vert (Success)
#4 et suivants	Vérification active	Vert ou Rouge selon l'état

7.5 DAG Airflow – Capture d'écran

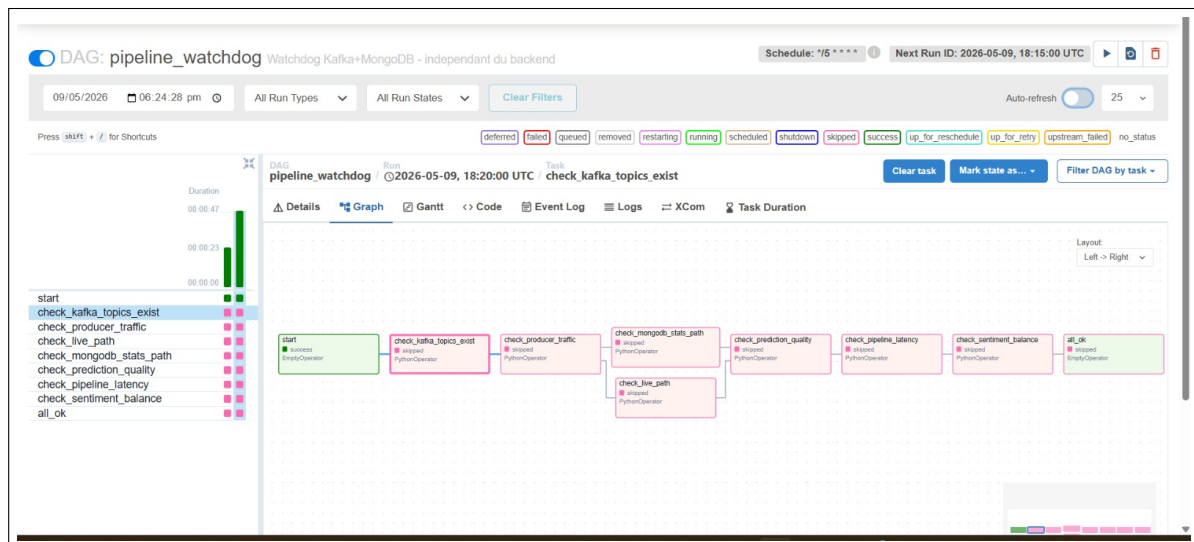


Figure 7.1: DAG pipeline_watchdog dans l'interface Airflow – Vue Graph

La capture montre le DAG `pipeline_watchdog` en cours d'exécution (09/05/2026, 18:20:00 UTC), planifié toutes les 5 minutes (* / 5 * * * *). On observe le graphe de dépendances des 9 tâches en vue *Left → Right*. Les tâches en rose indiquent l'état `skipped` (`AirflowSkipException`) : le pipeline n'était pas encore actif lors de ce run, ce qui est le comportement attendu. La tâche `start` apparaît en vert (`EmptyOperator` toujours success), et `check_kafka_topics_exist` est sélectionnée (encadrée en bleu). Le panneau de gauche liste les 9 tâches avec leur historique d'exécution (barres de durée).

7.6 Installation et lancement

```
# 1. Installer dans le venv existant
source venv/bin/activate
pip install apache-airflow==2.9.0 pymongo

# 2. Initialiser la base de données Airflow
export AIRFLOW_HOME=/home/mohamed/amazon-reviews-bigdata/airflow
airflow db init

# 3. Créer l'utilisateur admin
airflow users create --username admin --password admin123 \
  --firstname Mohamed --lastname Bouazza \
  --role Admin --email admin@example.com

# 4. Lancer le scheduler et le webserver
airflow scheduler &
airflow webserver --port 8080
```

Listing 7.2: Installation et démarrage d’Airflow

L’interface Airflow est accessible sur **http://localhost:8080**. Le DAG pipeline `_watchdog` s’active automatiquement et s’exécute toutes les 5 minutes, indépendamment de l’état du backend Django.

Conclusion

Ce mini-projet a permis de concevoir et déployer un pipeline de traitement de données massives complet, allant de l'ingestion des données brutes jusqu'à la visualisation interactive des prédictions en temps réel.

Sur le plan **Machine Learning**, le modèle de régression logistique entraîné avec Spark MLlib sur un pipeline TF-IDF atteint un **F1-macro de 0.8320** sur le jeu de test, avec une très bonne performance sur les classes Positif (0.93) et Négatif (0.84). La classe Neutre, intrinsèquement ambiguë et sous-représentée, constitue la principale marge de progression pour de futures itérations.

Sur le plan **Big Data**, l'architecture mise en place illustre concrètement les principes du streaming en temps réel :

Points clés de l'architecture

- ✓ **Découplage** : Kafka comme bus de messages entre Spark et Django
- ✓ **Dual path** : visualisation live (latence < 2s) et analyses statistiques batch
- ✓ **Scalabilité** : Spark Structured Streaming + MongoDB pour les flux massifs
- ✓ **Temps réel** : WebSocket pour le push browser sans polling
- ✓ **Supervision** : Apache Airflow comme watchdog externe indépendant (7 checks toutes les 5 min)
- ✓ **Containerisation** : Docker Compose pour le déploiement reproductible

L'ajout d'**Apache Airflow** constitue une valeur ajoutée significative : le DAG `pipeline_watchdog` surveille passivement Kafka et MongoDB sans aucune dépendance au backend, détectant en temps quasi-réel les pannes silencieuses (producer arrêté, Spark lag, qualité dégradée des prédictions). C'est l'approche standard d'observabilité en production.

En perspective, plusieurs améliorations peuvent être envisagées : l'intégration d'un modèle de *deep learning* (BERT, DistilBERT) pour améliorer la classification de la classe Neutre, la mise en place d'un cluster Kafka multi-broker pour la haute disponibilité, et l'ajout d'alertes automatiques sur les produits présentant un taux anormalement élevé d'avis négatifs.

Ce projet illustre concrètement comment les technologies Big Data modernes permettent de transformer un flux de données brutes en intelligence exploitable en quelques secondes, ouvrant la voie à des systèmes de veille commerciale en temps réel à grande échelle.